

The Four Layers: Prompt, Context, Harness, Loop — Aasan Summary (Roman Urdu + English)

Source: *The AI Agent Factory* — “The Four Layers: Prompt, Context, Harness, Loop” (Panaversity)

🔗 Bunyadi Idea (Wahi Story Jo Sab Ke Sath Hoti Hai)

Ek agent 40 minute chalta hai. \$50 kharch hote hain. Kuch bhi kaam ki cheez nahi banti. Aap log padhte ho — same 3 cheezein baar baar try hui hain, sirf alfaz thore alag.

Aap kya badalte ho? Zyada tar log **prompt** badalte hain — kyunki yeh sabse pehli, sabse aasan cheez hai edit karne ke liye.

Lekin prompt theek thi. Us poore system mein **koi bhi cheez yeh notice karne wali nahi thi ke run progress nahi kar raha** — isliye kisi ne isay roka nahi. Fix sirf **11 alfaz ka tha, ek doosri file mein, ek aisi layer par jiska naam zyada tar logon ko pata hi nahi.**

Yeh course aapko woh naam deta hai. **Chaar layers hain: Prompt, Context, Harness, Loop.** Yeh 4 skills nahi jinme se choose karo, aur na hi 4 steps jo chadhna hai. Yeh **4 containers hain, ek dusre ke andar**, aur har ek alag kaam ka zimmedar hai.

Ek dafa aap yeh dekh lo, “**mera agent kharab hai**” ek address wale sawal mein badal jata hai: **konsi layer tooti?**

🧠 Yeh Alfaz Kyun Confusing Hain

Yeh ek naya field hai, aur vocabulary **galat order mein aayi**. Pehle sirf ek layer thi jise chhu sakte the — aap type karte, jawab padhte, bas. **“Prompt engineering” poori skill ka naam thi.**

Phir tools bahar tak phaile — coding agent ne permissions file, rules file, hooks di (yeh ek layer hai). Phir schedules/routines aaye (yeh ek aur layer hai). **Do naye surfaces thore hi arse mein.**

Vocabulary saath nahi chal saki. Har surface ka naam jisne pehle likha usne rakh diya, aur **naam takra gaye**. Sabse zyada masla lafz “**harness**” karta hai — zyada tar writing isay “harness + loop” dono ke liye use karti hai, jaise woh ek hi layer ho.

Yeh sirf alfaz ki behas nahi: Ek missing permission rule aur ek missing schedule — **do alag bugs hain, alag surfaces par, alag log fix karte hain.** Ek lafz dono ke liye aapko **galat jagah** dhoondne bhej deta hai — bilkul wahi jo shuru ki kahani mein hua.

📖 Zaroori Alfaz (Ek Dafa Padho, Baad Mein Lookup Karo)

Term	Aasan Matlab
Prompt	Woh message jo aap bhejte ho
Context Window	Sab kuch jo model ek response likhte waqt dekh sakta hai
Curator	Jo decide karta hai window mein kya jaye, kis order mein, kya chhoote
Beat	Agent ka ek pura turn — instruction se lekar jab tak chup na ho jaye
Harness	Code jo model ke around hota hai, ek beat chalata hai
Loop	System jo beats start karta hai, unhe judge karta hai, beats ke beech yaad rakhta hai
Heartbeat	Jo beat start karta hai — schedule, event, condition
Spine	Model se bahar saved state, taake agla beat pichle ka pata rakhe
Stopping Condition	Testable rule jo bataye kaam khatam hai — maker ke alawa kisi aur ne choose kiya
Maker-Checker	Ek agent kaam karta hai, dusra check karta hai
Human Gate	Jahan insaan faisla kare, run aage barhne se pehle
Sub-agent	Apni khud ki window wala helper, ek kaam karta hai, summary wapis deta hai
Graph	Kai stacks ek sath wired — kya agla chalega, state kaise move hoti hai, kaun kis ko check karta hai

🔍 Poori Tasveer: 4 Containers, Ek Doosre Ke Andar

“Ek achi prompt bad context ke andar fail hoti hai. Achi context bare harness ke andar fail hoti hai. Achi harness bina loop ke khali baithti hai.”

Yehi is course ka core sentence hai — 3 dafa dohraya jayega. Matlab: **“ek doosre ke andar”** ka matlab — koi bhi andar wali layer aapko bahar wali layer ki kami se nahi bacha sakti, aur koi bahar wali layer andar wali kharab layer ko theek nahi kar sakti.

90 Second Ka Test (Isay Khud Try Karo)

Apna agent kholo. Ek chota, real task do jiska result aap check kar sakte ho. Poocho:

Fix this so it works, then tell me when you're done.

Ab, jawab par trust karne se pehle, **khud jakar check karo**. Script chalao, column add karo, link kholo.

Aksar aapko milega: ek confident **“done, all fixed”** jo kaam ke sath attached hai jo **kabhi verify hi nahi hua**. Yeh isliye kyunki **us setup mein kuch bhi check karne ke liye zaroori nahi tha, aur model ke alawa kisi ne decide nahi kiya “done” ka matlab kya hai.**

📦 Part 1: Shape

Concept 1 — Containers, Steps Nahi

Sabse aam tareeqa yeh 4 alfaz draw karne ka hai **seedhi ke jaisa**: beginners ke liye prompt neeché, experts ke liye loop upar. Yeh tasveer chupke se promise karti hai ke ek dafa aap acha ho gaye, purane steps peeche chhoot jate hain.

Yeh tasveer galat hai, aur is se paisa barbaad hota hai.

Har beat abhi bhi ek prompt banata hai. Ek loop jo 6 mahine se bina supervision chal raha hai, schedule par, checker aur spine ke sath — woh **abhi bhi din mein kai baar model ko message bhej raha hai**. Agar woh message vague hai, loop vague kaam **tezi se, ek timer par** banata hai.

3 galat baatein is tasveer se aasani se nikalti hain: - **Bahar ka matlab baad mein nahi.** Yeh order mein nahi banate — aksar 3 layers **rent** karte ho, ek **own** karte ho - **Bahar ka matlab zyada zaroori nahi.** Ek careless prompt, ek khubsurat engineered loop ke andar, **schedule par bura kaam** banata hai - **Yeh 4 sizes ka ek hi box nahi** — 4 alag alag cheezein hain

Concept 2 — Unit-of-Work Test

Har layer us **kaam ke tukde** se define hoti hai jiski woh zimmedar hai:

Layer	Unit of Work	Aasan Alfaz Mein
Prompt	Ek model call	Jo aapne type karke enter dabaya
Context	Window	Sab kuch jo model ek dafa dekh sakta hai
Harness	Ek beat	Instruction se lekar jab tak agent chup na ho jaye
Loop	Poora run	Jab koi type nahi kar raha, kya hota hai

Woh sawal jo alfaz badalne ke baad bhi zinda rehta hai: Jab koi “harness” ya “context engineering” ya “the agent loop” bole, lafz par behas mat karo. Yeh poocho: **“kya unit of work discuss ho raha hai — ek model call, window, ek beat, ya poora run?”**

🔍 Part 2: Charon Layers, Ek-Ek Karke

Concept 3 — Prompt (Unit: Ek Model Call)

Prompt woh message hai jo aap likhte ho: model kaun bane, aapko kya chahiye, achi kaam ka shape kya ho, examples, format. **Yeh sabse zyada over-invest hone wali layer hai** — kyunki isay bina code likhe practice kar sakte ho, aur 10 second mein edit kar sakte ho. Dono achi baatein hain, lekin yeh **trap** ban jati hain jab kuch aur production mein kharab ho.

Sahi nishani ke saath prompt hi masla hai: Model ne task samjha, roughly sahi kaam kiya, lekin jawab **galat shape, galat length, galat voice mein** aaya. Kuch factually galat nahi hai — bas woh nahi jo aapne manga tha.

Concept 4 — Context (Unit: Window)

Window woh sab kuch hai jo model ek response likhte waqt dekh sakta hai. **Ek direction mein isay literally lo:** file attach nahi ki, to woh model ke liye fact nahi. **Doosri direction mein literally mat lo:** window patli ho to model khali nahi ho jata — training se seekhi hui cheezein gap fill kar deti hain, **same confidence ke sath jaise woh sach ho.**

Curator hamesha exist karta hai (chahe aapne likha ho ya nahi), aur uske 3 kaam hain: 1. **Order** — position se accuracy badalti hai (“lost in the middle” — beech mein rakhi hui cheez kam yaad rehti hai) 2. **Compression** — 40 pages ko 4 mein summarize karna — jo cheez chhoot gayi, woh **wapis nahi aa sakti** 3. **Dropping** — jab window bhar jaye, kya nikale — agar aap policy nahi set karte, **harness aapke liye galat waqt par set kar dega**

Curator Test: Kisi document ki taraf point karo aur poocho “**kaunsi rule ne isay yahan rakha?**” Agar jawab “retriever ne wapis diya” hai, aapke paas ek search box hai, curator nahi.

Nishani jab context hi masla hai: Jawab fluent, confident, **factually galat**. Aksar **kisi aur cheez ke baare mein sahi** — purani file, dusra customer. Confident + wrong + kisi sach ke qareeb = **almost ek signature.**

Concept 5 — Harness (Unit: Ek Beat)

Aap ek instruction dete ho. Agent 3 files padhta hai, command chalata hai, error padhta hai, edit karta hai, dubara chalata hai, phir chup ho jata hai. Aapne ek dafa type kiya, **taqreeban 12 cheezein hui**. Yeh poori hissa **ek beat** hai, aur harness woh code hai jo isay chalata hai.

Zaroori surprise: Sub-agents ek tool call ki tarah bulaye jate hain, lekin kaam bohat bare ki tarah karte hain. Ek normal tool call result deta hai. **Ek sub-agent apni khud ki window kholta hai aur apna khud ka beat chalata hai.** Jo wapis aata hai, poori nested stack ka output hai.

Fayda: 40 documents sub-agent padh sakta hai, aapki window mein kabhi nahi aate, sirf 3 paragraphs milte hain. **Lekin nuksaan bhi real hai:** jo wapis aata hai woh ek summary hai, **poori confidence ke sath likhi hui — un cheezon samet jo sub-agent ne galat samjhi.** Aapne 40 documents nahi dekhe, koi aur bhi nahi dekhega.

Concept 6 — Woh Limit Jo Harness Ko Define Karti Hai ★

Yeh course ka turning point hai.

Ek beat kai wajah se khatam ho sakta hai: timeout, token ceiling, error, permission denied, ya model khud decide kare “done.” **Inme se koi bhi prove nahi karta ke kaam successful hua** — sirf yeh prove karta hai ke beat khatam hua.

Ek achi harness beat ke andar real checks enforce kar sakti hai (test suite chalana, schema validate karna). **Yeh model ka opinion nahi, verification hai** — jahan mumkin ho, banao.

Lekin boundary yaad rakho:

“**Ek beat prove kar sakta hai ke ek specific check pass hui. Yeh decide nahi kar sakta ke woh check pass karna kaafi tha.**”

Ek passing test sirf yeh prove karti hai ke woh test pass hui — yeh prove nahi karti ke test ne asal case cover kiya. **Kisi ko pehle se decide karna hai “finished” ka matlab kya hai.**

Bank reconciliation example: Agent 40 matches propose karta hai, kehta hai statement balance ho gaya — lekin kabhi dono totals compare nahi karta. Har internal signal healthy dikhta hai. **Claim phir bhi jhoot hai.**

Prompt mein “verify your work” likhna is masle ko theek nahi karta — model “verified” utni hi aasani se likh sakta hai jitna “done.” **Success ki final definition kaam se bahar se aani chahiye — isliye yeh agli layer ki zimmedari hai.**

Concept 7 — Loop (Unit: Poora Run)

Loop woh cheez deta hai jo ek beat khud nahi de sakta:

- **Heartbeat** — har beat start karta hai (schedule, event, condition)
- **Spine** — model se bahar state store karta hai, taake agla beat pichle ka pata rakhe

run: nightly reconciliation, 2026-03-14
done: pulled 412 payments and 388 open invoices
in progress: matching pass 3 of 5, 341 matched so far
needs a person: invoice 4471, two candidates both at 0.52
budget: 3 beats used of 12

- **Outside Stops** — poora run continue kare ya nahi:
 - **Success condition** (pehle se choose ki hui, command se prove ki hui)
 - **Limits** (beats, spending, time)
 - **No-progress check** (repeated attempts, koi meaningful change nahi)
 - **Separate checker** (jo kaam banata hai, wahi judge nahi karta)

Inme se koi bhi maker se nahi poochta “kya woh finished hai.” Yehi maker-checker rule ek sentence mein hai. **Shuru ki kahani (40 minute, \$50) ki wajah: no-progress check missing thi, behtar prompt nahi.**

Concept 8 — Human Gate: Ek Exit, Stop Nahi

Stops woh tareeqay hain jin se run safely fail ho. Gate woh tareeqa hai jis se run help ke sath succeed ho.

Zaroori idea: Ambiguity ek error nahi hai. Invoice 4471, 2 payments se match hoti hai, dono 0.52 score karte hain. Yeh matcher ka bug nahi — **data aisa hi hai.**

Bina gate ke, agent ke paas sirf 2 options hain, dono bure: 1. **Fail** — 9 passes ka legitimate kaam waste 2. **Guess** — **△ sabse khatarnak wala!**

Crash loud hoti hai. Guess silent hai — bilkul correct answer jaisa dikhta hai (same format, same confidence). Koi neech wala bata nahi sakta farq. **Ek fail hua run woh hai jo aap fix karte ho. Ek guessed run spreadsheet mein ek number hai jo chupke se galat hai.**

Gate pehle se likha jata hai, waqt par decide nahi hota. Trigger: confidence limit se neech, value limit se upar, ya koi bhi action jo undo karna mushkil ho. Jab fire ho, case **named person** ke paas jata

hai — unka jawab **wapis naye evidence ki tarah aata hai, run continue hota hai.**

🔧 Part 3: Map Use Karna

Concept 9 — Konsi Layer Toot Gayi?

Jo Aap Dekhte Ho	Pehle Kahan Dekho	Kya Badlo
Output shape/tone galat, lekin samjha task	Prompt	Sabse weak ingredient
Jawab confident, fluent, factually galat	Context	Curator — kya andar gaya, kya order, kya drop hua
Success report karta hai jo demonstrate nahi kiya	Harness	Tools, error handling, kya prove karna zaroori tha
Galat jawab bina check hue insaan/system tak pahuncha	Loop	Checker — kisne criteria choose ki
Kabhi rukta nahi, jaldi rukta hai, ya guess kiya jab poochna chahiye tha	Loop	Stops aur gate

Yeh ek verdict nahi, search order hai. Failures layers cross karte rehte hain. **Yeh table jis habit ko todne ke liye hai:** Team galat layer debug karti hai — kyunki prompt 10 second mein edit hoti hai, dabao mein log usay pakadte hain jo aasan hai, us cheez ki bajaye jo waqai tooti thi. **Layer ko zor se naam lena — pehle, kuch chhuane se pehle — पूरी discipline hai.**

Concept 10 — Aap Kaunsi Layers Waqai Own Karte Ho?

Layer	Mode 1 (general agent se problem solve)	Mode 2 (worker manufacture)
Prompt	Zyada tar aapka	Aapka, ek dafa likha, reuse hota
Context	Aadha aapka — baaki platform ka	Aapka — curator aap likhte ho
Harness	Rented, partly configurable	Aapka
Loop	Zyada tar rented	Aapka — har stop aapne likha

⚠ **Sabse Bara Galat Faisla:** “Main pehle se yeh sab karta hoon, kyunki mera Claude Code sab karta hai.” **Aapka tool apna kaam protect karta hai, na ke us worker ka jo aap deploy karne wale ho.** Claude Code apne beats cap karta hai — woh worker ke loop ko cap nahi karta jo aap banate ho. **Yehi gap hai jahan demo production incident ban jata hai.**

Concept 11 — Graphs Kahan Fit Hote Hain

Graph 5vi layer nahi hai. 4 layers ek execution path describe karti hain (ek message, ek window, ek beat, ek run). **Graph ek topology describe karta hai:** kya agla chalega, kya move hota hai, kaun kisay check karta hai.

4 layers batati hain ek node ke andar kya hota hai. Graph batata hai nodes ke beech kya hota hai.

Node **agent hona zaroori nahi** — function, rule, tool call, human gate, measurement bhi ho sakta hai.

6-node example (Accounts Payable): Pull (function) → Route (rule) → Match (**yeh agent hai — 4 layers isi ke andar hain**) → Gate (insaan, agar zaroori ho) → Prove (measurement — “isse koi loop bahas nahi kar sakta”) → Post (function). **Sirf ek node agent hai.**

Anthropic ne report kiya: single-agent research system ne chat ke muqable **~4x tokens** use kiye, multi-agent ne **~15x**. **Agent-heavy graphs ko apna cost justify karna padta hai.**

Concept 12 — Jab Framework Khud Se Ladta Hai

Ek map jise aap kabhi challenge na karo, ritual ban jata hai. 4 zaroori honest baatein:

1. **Kuch failures 2-layer hoti hain** — dono ho sakti hain, jo sasti fix mile wahi lo
2. **Kuch diagnoses ek layer par point karte hain, fix doosri layer par hoti hai** — yeh flaw nahi, yeh framework ka sahi kaam hai
3. **Kuch failures inme se koi bhi nahi hoti** — model genuinely task ke liye kaafi capable nahi. Naming layers guessing rokti hai, weak model ko strong nahi banati
4. **Har project ko 4 chahiye nahi** — ek dafa chalne wale task ko loop nahi chahiye

🔗 Yaad Rakhne Wala 1 Sentence

“Ek achi prompt bad context ke andar fail hoti hai. Achi context bare harness ke andar fail hoti hai. Achi harness bina loop ke khali baithti hai. Isliye jab kuch toote, fix se pehle layer ka naam lo.”

Yehi wajah hai ke demo mein chalne wala agent production mein fail hota hai. Demo ko sirf andar wali layers chahiye thin. **Bahar wali kabhi banayi hi nahi gayi thin. Model masla nahi tha. Uske around ki layers missing thin.**

📖 Quick Glossary Recap

Term	Aasan Matlab
Unit of Work Test	Sawal jo alfaz badalne ke baad bhi kaam karta hai: “ek call, window, beat, ya poora run?”
Curator Test	Kisi document ki taraf point karo, poocho “kaunsi rule ne isay yahan rakha?”
Outside Stop	Success condition jo maker set nahi karta — kisi aur ne enforce ki Error nahi — data ki asal shape, gate ka

Ambiguity	trigger
Guess vs Fail	Fail loud/fixable hai, guess silent/dangerous hai
Execution Graph	Kya agla chalega, state kaise move hoti hai
Governance Graph	Kaun kis ko check/approve/constrain karta hai

*Yeh summary poore “The Four Layers” crash course ka overview hai — poori series ka **naqsha (map)**. Loop Engineering, Harness Engineering, aur baaki courses is naqshe ki **territory** hain — woh cheezein banate hain jo yeh course sirf naam deta hai.*